Recursive Self-Improvement for Robot Policies: Recursion Improves, the Gate Protects, the Meta Level Repairs

Anonymous CVPR submission

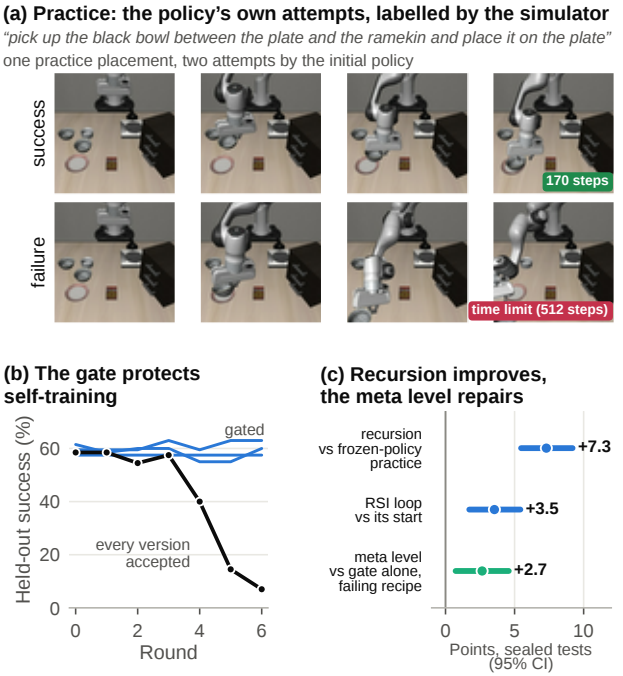


Figure 1. **Recursive self-improvement for a robot policy.** (a) One practice placement of a LIBERO-Spatial task and two attempts by the initial OpenVLA-OFT policy, as its camera saw them; the simulator's success check labels each attempt, and the loop trains on such labelled attempts. (b) LIBERO-Goal, behaviour cloning on the policy's own samples: when every new version is accepted, success on the round's 200 held-out placements falls from 58.5% to 7% within six rounds; under the gate the same recipe stays at its starting level. (c) On the sealed tests: the twelve-round loop against the same loop fed by the frozen initial policy and against its start (1,000 fresh scenes), and the language-model meta level against the gate alone on the failing recipe of (b) (official test).

Abstract

Recursive self-improvement (RSI) lets a robot policy get better without new demonstrations: the policy practises on fresh scenes, a verifier labels its attempts, it trains on them, a gate decides whether the new version replaces the old one, and a meta level may rewrite the training procedure itself. We built an RSI framework for vision-language-action (VLA) policies in which every level is a switch, and measured each level's contribution under a preregistered protocol with two sealed tests. With a 7B OpenVLA-OFT policy on LIBERO-Goal, each level does something training without it does not. Recursion improves: *after twelve rounds the loop is* $+3.5$ *points* $[+1.7, +5.4]$ *above its start on 1,000 sealed unseen scenes, while the same loop fed by the frozen initial policy falls below its start, so recursion is worth* $+7.3$ $[+5.5, +9.2]$. The gate protects: *a self-training recipe that collapses from 58.5% to 5.5% when every version is accepted stays at its starting level under the gate.* The meta level repairs: *from that recipe, a language model reading the loop's records diagnoses the fault in round one on every seed and lifts the loop* $+2.7$ *points* $[+0.7, +4.6]$ *over the gate alone, to the hand-tuned level. The limits are reported as registered: both margins miss our bars* ($+5$, $+3$)*, the meta level adds nothing to a working recipe, and one round of plain RL at equal compute is 2.7 points ahead of a strictly gated loop on unseen scenes. We also find that a published VLA-RL score owes* $+16.8$ *points to training on its test states. Code, preregistrations and run records are released.*

1. Introduction

Collecting demonstrations is the expensive part of training a robot policy. An attractive alternative is to let the policy teach itself: give it a task, let it try, keep the attempts that worked, train on them, and let the improved policy try again. For language models this recipe works in several forms [3, 4, 6, 19, 21–23]. Robot learning has started to follow: a vision-language-action (VLA) policy can be improved by RL in simulation from a success flag alone [12, 14]. Recursive self-improvement goes one step further and also lets the system revise the procedure by which it improves.

Two questions are rarely answered. First, *which part does the work?* A self-improvement loop bundles a verifier that labels attempts, a recursion in which the newest policy generates the next practice, a gate that keeps only versions

that beat their parent, and sometimes a meta level that edits the recipe; it also contains ordinary RL as a special case, so a positive result does not show that any of these mattered. Second, *is the gain real?* LIBERO [13] tests on a fixed list of initial placements, and nothing stops a method from training on that list. We show that a widely used VLA-RL checkpoint does, and that about a fifth of its score comes from it (Sec. 5.5).

We answer both by experiment. Our framework (Sec. 3) has the three levels of an RSI system, and each is a switch: *recursion* (practice generated by the latest policy, or by the frozen initial one), *selection* (a gate on fresh paired placements, or none) and a *meta level* that may rewrite the training recipe (off, a numeric search, or a language model that reads the loop's records), plus a verifier that can be shuffled and a plain-RL baseline that spends the whole budget in one round. Every comparison, bar and statistic was committed publicly before its runs, the official test and 1,000 freshly sampled test scenes were sealed and read once, and every final averages three evaluations (Sec. 4).

What we found. Each level contributes something that training without it does not (Fig. 1, Tabs. 2 and 3).

1. **Recursion improves the policy.** After twelve rounds the recursive loop is $+3.5$ points $[+1.7, +5.4]$ above its start on the sealed unseen scenes, on every seed; fed by the frozen initial policy it falls below its start, so recursion is worth $+7.3$ $[+5.5, +9.2]$ (Sec. 5.1).
2. **The gate protects it.** A recipe that collapses from 58.5% to 5.5% when every version is accepted stays at its starting level under the gate, and no gated run of 31 ends below its start beyond noise (Sec. 5.2).
3. **The meta level repairs the training procedure.** From the collapsing recipe, a language model reading the loop's records names the fault in round one on every seed and lifts the loop $+2.7$ $[+0.7, +4.6]$ over the gate alone, to the hand-tuned level; numeric search finds a repair on half the seeds and only in the last round (Sec. 5.3).
4. **The limits are real.** The improvement and repair margins miss our registered bars ($+5$, $+3$); on a recipe that already works the meta level adds nothing, because every recipe reaches the same plateau; and one round of plain RL at equal compute is 2.7 points ahead of a strictly gated loop on unseen scenes, because the gate discards rounds for noise (Sec. 5.4).
5. **A published VLA-RL score owes about a fifth of itself to its test states** ($+16.8$ points; Sec. 5.5).

Contributions. (i) An RSI framework for VLA policies in which recursion, selection and a language-model meta level are switches inside one engine, with BC and GRPO as interchangeable updates. (ii) A preregistered protocol with two sealed tests and public amendments. (iii) The first controlled measurement of what each level of RSI contributes on a VLA benchmark, including where it falls short and why. (iv) A measurement of train-on-test inflation in a published VLA-RL result. Code, preregistrations and all run records are released.

2. Related work

Self-improvement with verifiers. STaR [22], ReST [6] and ReST-EM [20] fine-tune a language model on its own verified rationales; SPIN [3] and self-rewarding models [21] close the loop with the model as its own judge; Absolute Zero [23] lets the model write its own tasks and uses a code executor as the verifier. RLVR with group-relative policy optimisation [4, 19] is the dominant update rule. These works report that the loop helps; we ask which part of it helps. The iterated on-policy analysis of [8] predicts the early saturation of a learner restricted to samples from the initial policy, which our non-recursive control shows at twelve rounds.

Meta levels that edit the procedure. LLM-driven search over programs and training recipes has produced strong results when the search space is large and evaluation is cheap [7, 15, 18]. Recent outer-loop systems put the weight on the harness around the proposer: Meta-Harness [11] hands it the code, scores and raw traces of every earlier candidate as files instead of a summary, EvoTrainer [2] adds layered diagnostics and a memory that keeps failed branches, and LLMZero [5] has agents diagnose training signals at checkpoints. Our last meta level follows them (Sec. 3.4). In our setting evaluation is expensive and noisy and the space is a few dozen knobs; there, a meta level that diagnoses correctly still cannot beat a fixed recipe that works, and we isolate the case where it can.

RL for VLA policies. OpenVLA [9] and OpenVLA-OFT [10] are the policies we improve; SimpleVLA-RL [12], RL4VLA [14] and related work [16] apply on-policy RL to such policies in simulation with a binary success reward. We use the SimpleVLA-RL training stack as one update operator and its released checkpoints as starting points. Our train-on-test measurement (Sec. 5.5) concerns the evaluation practice of this line of work rather than its methods.

Preregistration. Preregistered designs are standard in the experimental sciences [17] and have been proposed for machine learning [1]; we follow the practice literally, with every change a dated public amendment.

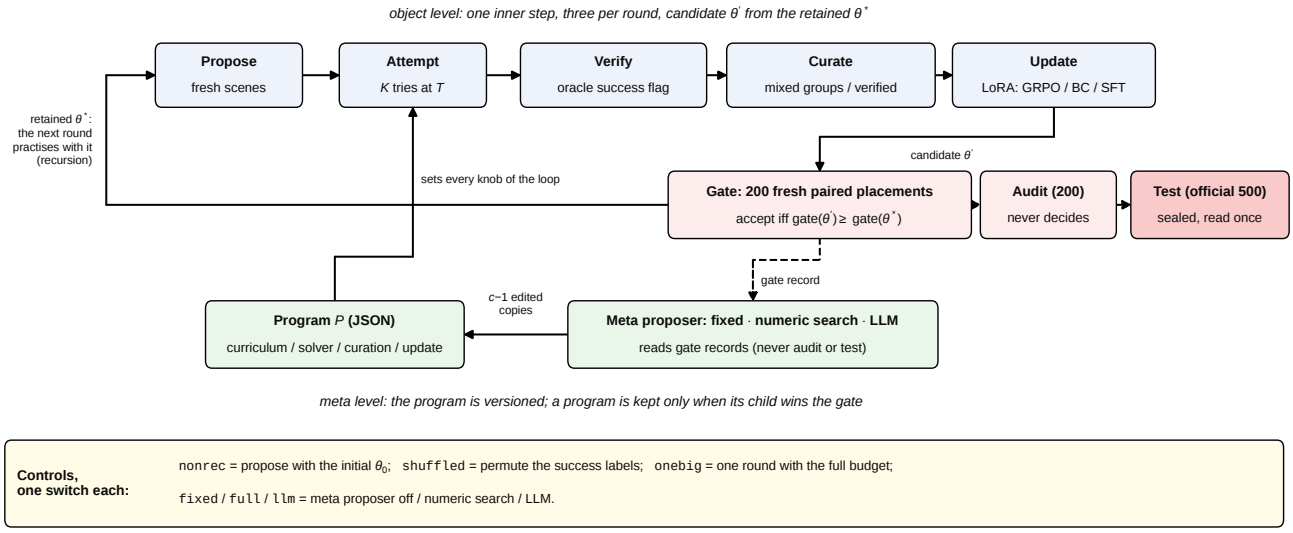


Figure 2. **The framework.** One engine, every ingredient a switch. The object loop practises on fresh scenes with the latest policy; the gate keeps a candidate only if it does not regress on paired held-out placements; the meta level versions the training program and may let a numeric search or an LLM edit it inside a declared space, seeing only gate records. Controls flip one switch each.

3. Framework

Figure 2 shows the framework. A policy practises on freshly sampled scenes, a verifier marks each attempt, the verified attempts update the policy, and a gate decides whether the updated policy replaces the current one. An optional meta level above may edit the training recipe.

3.1. Setting and notation

A policy π_θ acts in scenes $s \sim \mathcal{S}$ (object placements of a task); an attempt $\tau \sim \pi_\theta(\cdot \mid s)$ is a trajectory, and an external oracle $v(s,\tau) \in \{0,1\}$ the policy cannot influence labels it (the simulator's success flag). A *program* P is a JSON document with four sections—`curriculum` (which scenes to practise, how many, how allocated across tasks), `solver` (attempts per scene K, temperature T), `curation` (which attempts become training items) and `update` (the operator and its hyperparameters)—and $\mathcal{P}$ is a declared edit space of allowed values. A *version* is a pair (θ, P). The framework keeps an archive of versions; the one it currently trusts is the *retained* version $(\theta^\star, P^\star)$.

3.2. The loop (Algorithm 1)

One *inner step* runs program P from policy θ: propose scenes, attempt each K times, verify, curate, update. Curation `mixed_groups` keeps every attempt of scenes whose success rate lies strictly inside a band (used by GRPO [19]: the group-centred advantage is zero when all attempts agree); `verified` keeps successful attempts only (behaviour cloning, BC). The update is a LoRA step on θ. A *round* is three inner steps from the same parent, producing a *candidate* θ'. Because the next round starts from the retained version, practice is always generated by the latest policy: this is the *data recursion*, and the control `nonrec` breaks exactly it by proposing and attempting with the frozen initial policy θ_0 while still updating from $\theta^\star$.

3.3. Gate, audit and sealed test

Three disjoint evaluation sets are drawn per run from seeds that never overlap the official benchmark states. The *gate* (20 placements per task, 200 episodes, greedy decoding, environment re-seeded per episode so that versions are compared on identical placements) decides: a candidate is kept iff its gate success is at least the parent's on the same placements. In the *fresh-paired* mode used in every main run, a new gate set is drawn every round and the parent is re-evaluated on it, so acceptance is always a paired comparison on placements never seen before and a version cannot overfit a fixed gate. Because the gate's verdict decides the next version, and that version generates the next practice, the loop is recursive in selection as well as in data. The *audit* (200 more episodes) is recorded but never decides anything; it exposes gate overfitting. The *test* is the official benchmark (50 initial states per task, 500 episodes) and is *sealed*: no process in the loop can read it, and it is opened once, after every decision in every run of a study has been made. Because the loop practises on sampled placements while the official states are a fixed list, a second sealed test, registered before any of its evaluations, scores each retained policy on 1,000 freshly sampled placements that no run trained, gated or audited on.

Algorithm 1 Gated recursive self-improvement with a meta level

Require: initial policy θ_0, seed program P_0, oracle v, gate set size n_g, rounds R, candidates per round c

$(\theta^\star, P^\star) \leftarrow (\theta_0, P_0)$; evaluate $\theta^\star$ on fresh gate and audit sets
for $r = 1 \dots R$ **do**
  draw a fresh gate set G_r of n_g paired placements; re-evaluate $\theta^\star$ on G_r
  $\{P_1, \dots, P_c\} \leftarrow \{P^\star\} \cup$ PROPOSE(round records, $\mathcal{P}$) ▷ meta level: fixed / numeric / LLM
  for each candidate program P_i **do**
    $\theta_i \leftarrow \theta^\star$
    for inner step $= 1 \dots 3$ **do**
      scenes $\leftarrow$ PROPOSE$(P_i, \theta_{\text{gen}})$ with $\theta_{\text{gen}} = \theta_i$ (recursive) or θ_0 (`nonrec`)
      attempts $\leftarrow K$ samples per scene at temperature T; labels $\leftarrow v(\cdot)$ (or permuted: `shuffled`)
      items $\leftarrow$ CURATE(attempts, P_i); $\theta_i \leftarrow$ UPDATE(θ_i, items, P_i)
    end for
    $g_i \leftarrow$ success of θ_i on G_r (greedy); record audit, never use it
  end for
  $i^\star \leftarrow \arg\max_i g_i$; **if** $g_{i^\star} \geq g(\theta^\star, G_r)$ **then** $(\theta^\star, P^\star) \leftarrow (\theta_{i^\star}, P_{i^\star})$ ▷ gate
end for
read the sealed test of $\theta^\star$ once, after every run of the study has ended

3.4. Meta level

The meta level versions the program and lets it be improved under the same gate. Each round the incumbent $P^\star$ and $c-1$ proposed programs are each trained for real from the same parent, and the gate picks the version to keep; a program is credited only when its child wins. Three proposers:

- `fixed`: no change (the meta level is off).
- `numeric`: random mutation of declared knobs, biased toward knobs whose past changes raised the gate.
- `llm`: a language model (Claude Sonnet) reads the current program, the edit space with a neutral description of what each option computes, and, for every past candidate, its program diff, gate score, acceptance and per-step statistics (sampled success, share of mixed scenes, item counts, first/last training loss). It must write a diagnosis, then return one new program.
- `llm+harness`: the same model inside a harness, following [2, 11]: a *workspace* of files (the full record of every candidate of the run, per-task gate tables, an evidence ledger that pools gate changes per edited field), a frozen *memory* of earlier runs with a playbook the model distilled before launch, its own notes from earlier rounds, and list, read and search tools used over several turns. The prompt states the measured gate noise. Building it exposed a defect in the plain `llm` proposer: the training loss it was told to watch never reached it (a field-name error), in every run reported for it below.

Table 1. **Controls.** Each arm flips one switch of Algorithm 1.

arm	change in Algorithm 1	isolates
`fixed`	none (proposer off)	gated recursive loop
`nonrec`	$\theta_{\text{gen}} = \theta_0$	data recursion
`shuffled`	labels permuted per scene	the verifier
`onebig`	R=1, $3R$ inner steps	gate + rounds vs one-shot RL
`full`	numeric proposer, c=2	meta level (equal compute)
`llm`∗	LLM proposer, c=2	meta level (equal compute)

∗ `llm`: knob space; `llm_harness`: harness space; `llm_harness_cap`: harness space under the compute cap.

The *knob* space has ten hyperparameters (scenes, allocation, samples, temperature, curation rule and budget, operator, learning rate, steps, batch); the *harness* space adds nine components of the procedure (policy-gradient ratio, loss aggregation, BC likelihood temperature, advantage normalisation, curation band, per-scene cap, allocation floor, two clip bounds). Each corresponds to a change made by hand during development after diagnosing round records; the LLM is never told which option is better, so rediscovering them is a test of the meta level.

Attribution guard. The meta level never sees the audit, the test, the weights or the episodes; it only chooses values inside $\mathcal{P}$, every proposal passes the same validator as a numeric mutation, failures fall back to numeric search and are counted, and an optional *compute cap* keeps any candidate within the starting program's rollout and optimiser budget. All practice data comes from the policy's own attempts and all verdicts from the oracle; the LLM chooses procedure, not data.

3.5. Every control is a switch

Table 1 lists the arms used in the experiments. Each differs from `fixed` in exactly one line of Algorithm 1, so a difference between an arm and `fixed` is attributable to that line; `full` and the `llm` arms train the same number of candidates and so cost the same.

4. Experimental setup

4.1. Benchmarks, policies and starting points

LIBERO / OpenVLA-OFT. LIBERO-Goal [13] has ten goal-conditioned tasks in one scene; LIBERO-10 ten long-horizon tasks. The policy is OpenVLA-OFT [10] (7B, action chunks of 8), started from a supervised fine-tune on *one* demonstration per task [12] so that there is room to

improve (about 59% on LIBERO-Goal, 19% on LIBERO-10). Rollouts and GRPO/BC training use the SimpleVLA-RL stack with LoRA (r=32). The seed program—chosen from sealed pilots and frozen by amendment before any main run—is GRPO with learning rate 3×10^{-5}, matched-ratio log-probabilities, episode-balanced loss, 8 samples per scene at temperature 1.5, 160 scenes per inner step, 200 optimiser steps of batch 64; 6 rounds of 3 inner steps; fresh paired gates.

4.2. Arms and baselines

The LIBERO-Goal main study runs the eight arms of Sec. 3.5 with three seeds each (a fourth seed was added to the `nonrec`, `shuffled`, `onebig` and `fixed` arms by amendment before any of their runs ended), 31 runs in all, on two GPU platforms: a shared 8×A800 node and a 16-GPU cloud pool with its own evaluation stack. The stacks are not bit-identical, so every comparison is paired within a platform and the platforms serve as replicates. `onebig` is the plain-RL baseline: the whole six-round rollout and update budget in one round of 18 on-policy inner steps from the initial policy, with the same sampling and update code; it is read both through the gate and as its ungated single candidate.

4.3. Preregistered protocol

We fixed the protocol before the runs and release the pre-registration documents and every dated amendment with the code.

Sealing. The official LIBERO test states are never used for training, sampling, gating or program selection, and no test file is opened before a study's read-once. Only two sealed pilots per model were allowed before the seed program was frozen.

Finals and reference. The *same* initial policy scored between 57.2% and 60.0% in different runs, and about 40% of episodes flip between nearby policies. Each run's final is therefore the per-episode mean of three evaluations of its retained policy, and the initial reference is the per-episode mean of every in-run evaluation of the initial policy on that platform (36 on the cloud pool, 21 on the A800). A noise check (the same statistic between two halves of the initial-policy evaluations) accompanies every verdict.

Claims and bars. Seed-averaged per-episode paired bootstrap (10,000 draws), 95% CI. "CI > 0" means the interval excludes zero.

Table 2. **Twelve rounds, two sealed tests** (LIBERO-Goal, cloud pool, seeds 1–3; points, 95% CI). Official: 500 fixed states, three-evaluation finals. Fresh: 1,000 sampled placements, disjoint from every practice, gate and audit set and registered before any evaluation on them.

comparison	bar	official test	fresh scenes
`fixed12` − start	+5	+1.60 [−0.88, +4.13]	+3.54 [+1.72, +5.42]
`fixed12` − `nonrec12`	+2	+7.84 [+5.22, +10.49]	+7.30 [+5.47, +9.20]
`fixed12` − `onebig12`	+2	−0.38 [−2.42, +1.64]	−2.73 [−4.30, −1.20]

claim	comparison	bar (points)
R1 improvement	`fixed`, `full` − reference	$\geq +5$, CI > 0, 2/3 seeds > 0
R2 verifier	`fixed` − `shuffled`	$\geq +3$, CI > 0
R3 recursion	`fixed` − `nonrec`	$\geq +2$, CI > 0
R4 meta level	each meta arm − `fixed`	$\geq +2$, CI > 0
R5 vs one-shot RL	best gated arm − `onebig`	$\geq +2$, CI > 0
M2-A meta v2	`llm` − identical-recipe replica	$\geq +3$, CI > 0
S1 self-repair	`llm_harness` − `fixed` (failing start)	$\geq +3$, CI > 0

Negative readings are reported as such; the reading scripts were committed before each read and their outputs are released.

5. Results

We report what each level of the architecture contributes (Fig. 2): recursive practice (Sec. 5.1), the gate (Sec. 5.2) and the meta level (Sec. 5.3), then how the loop compares with plain RL (Sec. 5.4). Every number comes from a preregistered read of a sealed test unless it is marked *gate-level*.

5.1. Does recursive self-improvement make the policy better?

Finding 1: recursive self-practice improves the policy on unseen scenes, and recursion is what makes it work. A registered twelve-round study (three seeds each of `fixed12`, `nonrec12` and a 36-step `onebig12`) was read on both sealed tests (Tab. 2). On 1,000 fresh scenes that no run trained, gated or audited on, the recursive loop ends $+3.5$ points $[+1.7, +5.4]$ above its start, on every seed; the registered bar was $+5$ and is missed. The same loop with practice generated by the frozen initial policy ends 3.8 points *below* its start, so recursion is worth $+7.3$ $[+5.5, +9.2]$ on the fresh scenes and $+7.8$ $[+5.2, +10.5]$ on the official test, positive on every seed and well over its $+2$ bar on both. The mechanism is visible in the records: once the learner has moved, practice generated by the initial policy no longer matches it, its candidates are truly a little worse than their parent, and with acceptance on $\geq$ noise-driven acceptances accumulate, although every `nonrec12` version passed a fresh paired gate. Recursive practice keeps the data on the learner's own distribution, which is the point of letting the system generate its own practice.

Finding 2: at six rounds the gains are small, and verified, recursive practice is where they come from. Every gated run ends within noise of its start or above it (finals

Table 3. **Registered claims on LIBERO-Goal** (official 500-episode test, 3-evaluation finals, pooled initial reference, seed-averaged paired bootstrap). Platform Y = cloud pool, A = A800.

claim	comparison	points	95% CI	bar	seeds
R1	`fixed` − ref. (Y)	+1.71	[−0.76, +4.23]	+5	1–3
R1	`full` − ref. (Y)	+1.28	[−1.37, +3.99]	+5	1–3
R2	`fixed` − `shuffled` (Y)	+1.22	[−0.37, +2.85]	+3	0–3
R3	`fixed` − `nonrec` (Y)	+1.72	[−0.28, +3.68]	+2	0–3
R4	`full` − `fixed` (Y)	−0.42	[−2.40, +1.56]	+2	1–3
R4	`llm_harness` − `fixed` (Y)	−0.73	[−2.58, +1.07]	+2	1–3
R4	`llm` − `fixed` (A)	+1.04	[−0.84, +3.00]	+2	1–3
R4	`llm_harness_cap` − `fixed` (A)	+1.16	[−0.76, +3.16]	+2	1–3
R5	best gated − `onebig` (Y)	−0.58	[−2.12, +0.95]	+2	0–3
noise	half vs. half of the n0 evaluations (Y / A)	+0.64 / +0.54	[−0.07, +1.36] / [−0.24, +1.35]		

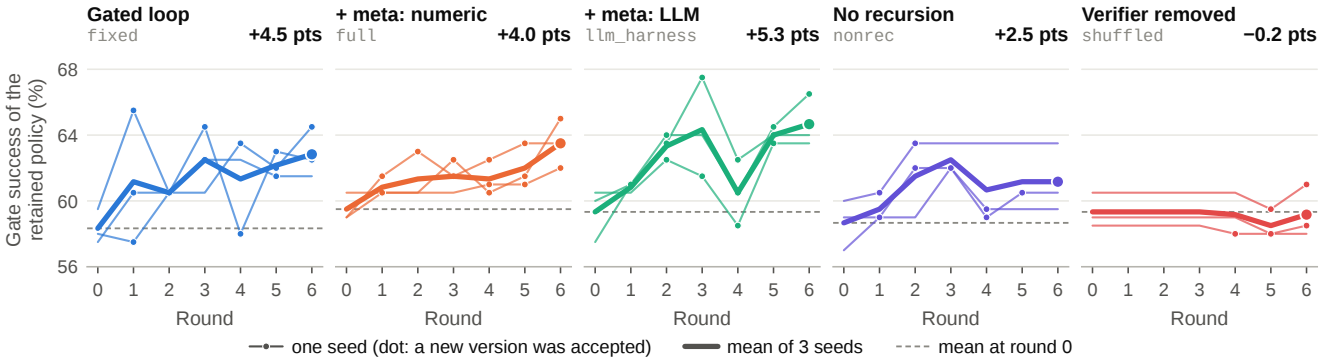


Figure 3. **Gate success of the retained policy by round** (cloud platform; thin lines are seeds 1–3, the thick line their mean). A dot marks a round that accepted a new version; every round is scored on 200 newly drawn placements, so a retained version's score can fall from one round to the next. The number in each panel is the change of the mean from round 0 to round 6. With a real verifier and recursive practice the loop climbs 4–5 points at the gate within six rounds; without the verifier (`shuffled`) it stays flat, and with practice from the frozen initial policy (`nonrec`) it stops after round 3. Gate gains overstate test gains by the winner's curse (Tab. 4).

Table 4. **Per-arm finals** (%, mean of the 3-evaluation finals over seeds 1–3; seed-0 runs listed separately). Reference 58.9 (Y) / 58.4 (A).

	`fixed`	`full`	`llm`∗	`nonrec`	`shuf.`	`onebig`
Y, s1–3	60.6	60.2	59.9	59.1	58.9	59.1
A, s1–3	59.1	–	60.1 / 60.2	–	–	–
A, s0	57.2	–	–	54.8	57.5	62.7

∗ Y: `llm_harness`; A: `llm` / `llm_harness_cap`.

57.1–62.2% against a reference of 58.4–58.9%; Tab. 4); the typical gain is one to two points, the best single seed gains +3.3 [+0.1, +6.6], and the registered improvement claim R1 comes to +1.71 [−0.76, +4.23] against a +5 bar (Tab. 3). The two largest ingredients are recursion (R3 +1.72 [−0.28, +3.68], just under its +2 bar) and the verifier (R2 +1.22): without a real verifier the loop ends where it started. At the gate (Fig. 3) the separation is visible within six rounds; at twelve it reaches the sealed tests (Finding 1).

5.2. Is self-training safe? The gate

Finding 3: the gate is what keeps self-training from collapsing. The behaviour-cloning recipe of Sec. 5.3 collapses from 58.5% to 5.5% on held-out placements when its versions are accepted unconditionally (Fig. 1b); under fresh paired gates the same recipe does not fall below its start beyond noise on the sealed test, and no gated run of the main study does. The gate is cheap (one extra evaluation per candidate) but noisy: a 200-episode paired comparison has a standard deviation of about 3.4 points, so it accepts and rejects many candidates by chance. Its protection relies on recursive practice: over twelve rounds a chain of gate-accepted `nonrec12` versions drifts below its start (Finding 1).

5.3. Can the system improve its own training procedure? The meta level

A recipe that fails. The meta level can be told apart from RL only where RL's recipe fails. One such case was found

Table 5. **Self-repair study**: gate / sealed-test final (%, three evaluations) of each retained version, and what changed. S1 = `llm_harness` − `fixed` = +2.65 [+0.73, +4.58] (bar +3 missed); S2 = `llm_harness` − `full` = +1.15 [−0.63, +2.92]; S3 = `llm_harness` − hand-tuned GRPO loop = +0.28 [−1.43, +1.97].

seed	`fixed` (gate only)	`full` (numeric)	`llm_harness`
1 (Y)	60.0 / 56.1	62.5 / 56.7	65.5 / 60.1, GRPO in r1
2 (Y)	start / 58.7, all refused	65.0 / 59.9, GRPO	68.5 / 61.3, GRPO in r1
3 (Y)	63.0 / 57.5	61.0 / 60.2, BC rollout-T	64.5 / 57.9, BC T 1.6→1.3
0 (A)	64.5 / 57.4	61.5 / 58.8	68.5 / 60.9, GRPO in r2

before any meta-level run: behaviour cloning on the policy's own temperature-1.6 samples with a unit-temperature likelihood collapses when its versions are accepted unconditionally. The repair is a one-knob change (the BC temperature, the sampling temperature or the update operator), and all three knobs are in the harness space, but the meta level is never told which. A registered study starts from this recipe with arms `fixed` (the gate alone), `full` (numeric search) and `llm_harness` (same compute), four seeds, and the bar S1 = `llm_harness` − `fixed` ≥ +3, CI above 0.

Finding 4: the meta level diagnoses and repairs a failing training procedure on its own. On all four seeds the language model's first-round diagnosis names the mismatch between the sampling and likelihood temperatures and the missing failure signal (verbatim in the released logs), and its first proposal is the same: switch to GRPO on mixed-outcome scenes at a lower temperature. The gate kept that switch in round 1–2 on three seeds and refused it twice on the fourth, where the model repaired BC instead (Tab. 5). The repaired loop then gains what the hand-tuned loop gains: S1 = +2.65 points [+0.73, +4.58] over the gate alone, positive on three seeds with the interval above zero, and level with the hand-tuned GRPO loop (S3 = +0.28 [−1.43, +1.97]), while the gate alone holds the policy at its start (−2.37 [−4.12, −0.62] against the hand-tuned loop). Two qualifications are registered. S1 is under its +3 bar, so the claim is not made in its registered form. And numeric search over the same knobs also finds a repair, on two seeds and only in round 6, ending 1.15 points behind ([−0.63, +2.92]): the model's advantage is speed and reliability of the diagnosis, not a better end point. Six gate-only pilots with extreme GRPO and BC recipes found no second failing scenario.

Finding 5: on a recipe that already works, the meta level finds nothing to repair. All four meta-level arms of the main study land within ±1.2 points of `fixed` (R4, Tab. 3). A second registered study ("v2") gives the model per-task gate rates with their noise and compares it with a *replica* arm that trains the identical recipe at the same compute: −0.27 [−2.53, +2.00] (cloud, seeds 1–3) and +0.30 [−1.60, +2.15] (A800, seeds 0–3). The records show why. The model's diagnoses are correct (an unsolved task gives GRPO no signal, another is saturated, the remaining gate changes are noise) and its edits are those a practitioner would try, but its proposals scored −1.2 points against the incumbent at the gate (n=30) where replicas scored +0.6 (sd 3.4, n=36): *every recipe in the space reaches the same plateau*, so there is nothing for a correct diagnosis to fix.

5.4. Against plain RL at equal compute

Finding 6: a strictly gated loop does not beat plain RL on a recipe that works. One round of plain RL with the whole budget (`onebig`) matches the six-round loop (R5 −0.58 [−2.12, +0.95]), ties the twelve-round loop on the official test (−0.38) and is 2.7 points ahead of it on the fresh scenes (Tab. 2). The cost is the gate's strictness: the twelve-round runs kept 6, 9 and 9 of their 12 rounds, a refused round's training is discarded, and most refusals in our records are dips within the gate's own noise. Training without a gate loses no rounds, but has no protection either: without it the BC recipe of Fig. 1b collapses.

5.5. How much of a published VLA-RL number is the test set?

Finding 7: about a fifth of the headline number. The released SimpleVLA-RL LIBERO-10 checkpoint was trained by RL on the 50 fixed test states per task. We evaluated it, and its own SFT start, on the 500 official episodes and on 500 placements freshly drawn by LIBERO's own sampler (the same distribution, not a perturbation): SFT 14.6% official against 20.0% fresh, RL 87.4% against 76.0%. The difference of the two gaps, which cancels any difference in difficulty between the placement sets, is +16.8 points [+9.8, +23.4]; 500 further never-seen placements give +16.0 [+9.2, +22.4] (Fig. 5). The method works (the checkpoint gains 56 points on fresh placements), but about a fifth of its headline number is memorisation of the test states.

6. Discussion

What the evidence does not support. Two registered bars were missed: the loop's gain over its start (+3.5 against +5) and the self-repair margin (+2.7 against +3); at six rounds no ingredient cleared its bar. We do not claim that the loop beats plain RL at equal compute, nor that the language model's diagnosis, rather than the knob space it edits, carries the repair (its margin over blind search is +1.2 [−0.6, +2.9]).

For practitioners. A fresh paired gate makes an unsafe recipe safe for one extra evaluation per candidate, but its strictness should match its noise. A single 500-episode evaluation carries about ±1.5 points of noise, so finals should

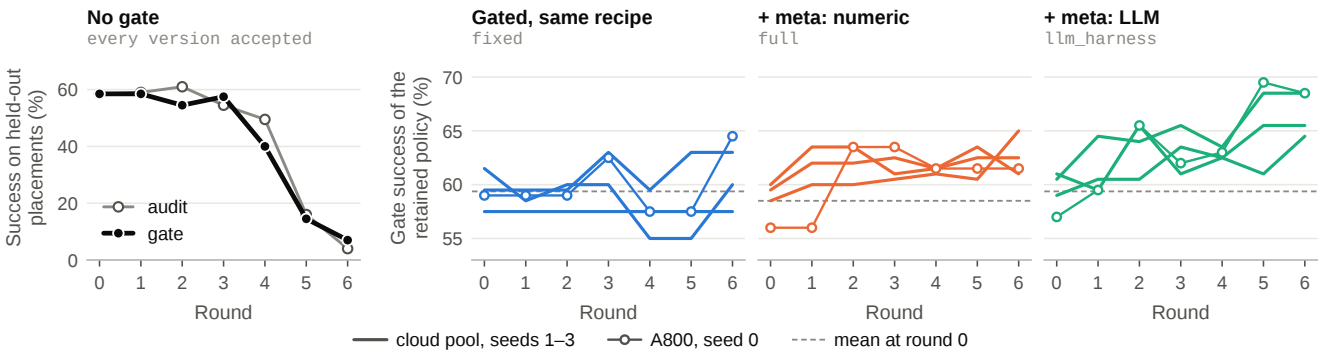


Figure 4. **Self-repair study, gate level.** Left: the starting BC recipe collapses when every version is accepted. Right (narrower scale): under the gate the same recipe stays at its starting level (`fixed`); both proposers find repairs; at the gate the LLM arm looks 4–8 points ahead, on the sealed test it is +2.65 (Tab. 5).

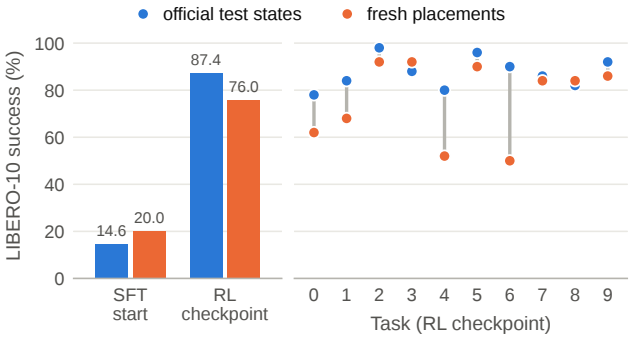


Figure 5. **Train-on-test inflation** (LIBERO-10). Left: official vs. fresh placements for the SFT start and the RL checkpoint trained on the official states. Right: the RL checkpoint per task; the loss is concentrated on tasks 4 and 6.

be averaged, and freshly sampled test scenes should complement a fixed list of test states. A language-model meta level pays off only where the recipe can fail.

Limitations. One simulated benchmark family, one 7B policy and a few-demonstration start; other policies and real robots are untested (real-robot data collection is in progress). Runs interrupted by hardware were continued from a completed round, as the amendments disclose. Preregistration constrains the analysis, not the design: a different recipe space might leave more for a meta level to find.

7. Conclusion

Each level of recursive self-improvement does a distinct job for a robot policy: recursion makes it better on unseen scenes, the gate keeps self-training from collapsing, and the meta level repairs a failing training procedure. The effects are modest at this budget, and we report every registered bar we missed; we hope the protocol (sealed tests, fresh scenes, averaged finals, public amendments, one read) outlives the numbers.

References

[1] Samuel Albanie, James Thewlis, Sébastien Ehrhardt, and João F. Henriques. On the preregistration of machine learning experiments. *NeurIPS Workshop on Preregistration in ML*, 2021. 2

[2] Guhong Chen, Yingcheng Shi, Yongbin Li, Binhua Li, Xander Xu, Hu Wei, Shiwen Ni, Min Yang, and Jieping Ye. EvoTrainer: Co-evolving LLM policies and training harnesses for autonomous agentic reinforcement learning. *arXiv:2606.03108*, 2026. 2, 4

[3] Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models. In *ICML*, 2024. 1, 2

[4] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv:2501.12948*, 2025. 1, 2

[5] Haoyang Fang, Wei Zhu, Boran Han, Alex Zhang, Zhenyu Pan, Shuo Yang, Shuai Zhang, Jiading Gai, Peng Tang, Cuixiong Hu, Xuan Zhu, Huzefa Rangwala, George Karypis, and Bernie Wang. LLMZero: Discovering adaptive training strategies for RL post-training via LLM agents. *arXiv:2606.18388*, 2026. 2

[6] Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, et al. Reinforced self-training (ReST) for language modeling. *arXiv:2308.08998*, 2023. 1, 2

[7] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *ICLR*, 2025. 2

[8] Juno Kim, Jihun Yun, Jason D. Lee, and Kwang-Sung Jun. Coverage improvement and fast convergence of on-policy preference learning. *arXiv:2601.08421*, 2026. 2

[9] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. OpenVLA: An open-source vision-language-action model. In *CoRL*, 2024. 2

[10] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv:2502.19645*, 2025. 2, 4

[11] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-Harness: End-to-end optimization of model harnesses. *arXiv:2603.28052*, 2026. 2, 4

[12] Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaoyang Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, et al. SimpleVLA-RL: Scaling VLA training via reinforcement learning. *arXiv:2509.09674*, 2025. 1, 2, 4

[13] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS Datasets and Benchmarks*, 2023. 2, 4

[14] Jijia Liu, Feng Gao, Bingwen Wei, Xinlei Chen, Qingmin Liao, Yi Wu, Chao Yu, and Yu Wang. What can RL bring to VLA generalization? an empirical study. *arXiv:2505.19789*, 2025. 1, 2

[15] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv:2408.06292*, 2024. 2

[16] Guanxing Lu, Wenkai Guo, Chaoyi Zhang, Yue Zhou, Haonan Jiang, Zifeng Gao, Yansong Tang, and Ziwei Wang. VLA-RL: Towards masterful and general robotic manipulation with scalable reinforcement learning. *arXiv:2505.18719*, 2025. 2

[17] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *PNAS*, 115(11):2600–2606, 2018. 2

[18] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv:2506.13131*, 2025. 2

[19] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv:2402.03300*, 2024. 1, 2, 3

[20] Avi Singh, John D. Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J. Liu, James Harrison, Jaehoon Lee, Kelvin Xu, et al. Beyond human data: Scaling self-training for problem-solving with language models. In *TMLR*, 2024. 2

[21] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. In *ICML*, 2024. 1, 2

[22] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *NeurIPS*, 2022. 2

[23] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv:2505.03335*, 2025. 1, 2